

CogniTek

Personalized AI-Powered Academic Assistant

✓ Production Ready · Submitted May 2026

1 · What Is This Project?

CogniTek is a full-stack, cloud-deployed academic assistant built for B.Tech students at Kerala Technological University (KTU). It transforms everyday study audio — lecture recordings, voice notes, revision narrations — into structured, actionable study data automatically, using a pipeline of industry-grade AI models.

The application is live on the web and accessible as a Progressive Web App (PWA) on Android and desktop. The live link is available in the GitHub repository description.

2 · Why This Is Cool — The Ear-Nerve-Brain-Memory Architecture

CogniTek is modelled after how the human cognitive system works, with four distinct layers that mirror biological processing:

Layer	Technology	What It Does
 EAR (Perception Layer)	Groq Whisper Large V3	Captures and transcribes audio input — lectures, voice notes, or auto-recordings — at blazing speed using Groq's Language Processing Unit (LPU).
 NERVE (Relay Layer)	FastAPI + Groq LPU	Routes the raw transcript to the correct processing pipeline. The Groq LPU delivers inference at a 1 : 300 speed ratio compared to a conventional GPU inference server — meaning what would take 300 seconds on a local model takes just 1 second here. This is why we do NOT fall back to a slow local model.
 BRAIN (Intelligence Layer)	Google Gemini 2.5 Flash	Analyses the transcript and extracts Actionable Tasks, Due Dates, Exam Schedules, Placement Milestones, and Flashcard Study Decks. Also powers Sylens, the context-aware AI companion chatbot.
 MEMORY (Persistence Layer)	Supabase PostgreSQL	Stores all student data — tasks, flashcards, profiles, exam sessions — securely in the cloud. Authentication is handled by Supabase Auth, giving each student an isolated private data space.

3 · System Requirements

The following versions were used and verified during development and deployment:

Component	Version	Purpose
Python	3.14.2	Backend runtime — FastAPI, Whisper, Gemini SDK
Node.js	v24.12.0	Frontend build toolchain — Vite + React
npm	11.6.2	Package manager for frontend dependencies
React	19	Frontend UI framework
FastAPI	Latest	Python web framework for the REST API
Tailwind CSS	v4	Utility-first CSS framework
FFmpeg	System PATH	Required by Whisper for audio decoding

4 · The Flex — Why Groq LPU and NOT a Local Model

A deliberate, performance-driven decision was made to use Groq's Language Processing Unit for transcription instead of running OpenAI Whisper locally on the server.

- **Speed — 1 : 300 Ratio:** Groq LPU processes audio at a 1 : 300 inference speed ratio over conventional GPU-based servers.
- **Real-world impact:** A 1-minute lecture recording is transcribed in under 1 second. The equivalent local inference takes 5–10 minutes on a mid-range machine.
- **Cost:** Whisper Large V3 on Groq costs \$0.111 / hour of audio — a 60-minute lecture = ~\$0.11. This is negligible for a college use case.
- **Scale:** Groq supports 1,000 requests/minute and 250,000 tokens/minute — more than enough for a college cohort.
- **Zero infrastructure overhead:** The local Whisper 'medium' model requires CUDA, FFmpeg, PyTorch, and 4–6 GB RAM. Groq requires only an API key.

⚠ Do not revert to a local Whisper model. The speed, reliability, and zero-maintenance advantage of Groq LPU is a core design decision, not an optional optimisation.

5 · Full Technology Stack

Frontend (The Interface)

- React 19 + Vite 7
- Tailwind CSS v4
- React Router v7
- Supabase JS Client — auth + real-time
- Capacitor — Android PWA wrapper
- Deployed on Vercel

Backend (The Brain)

- Python 3.14.2 — FastAPI framework
- SQLAlchemy ORM — PostgreSQL in production (Supabase), SQLite for local dev
- Groq API — Whisper Large V3 for transcription, Llama 3 for fast chat
- Google Gemini 2.5 Flash — task/flashcard/exam extraction, Sylens AI companion
- Deployed on Render.com

Cloud Services

- Supabase — PostgreSQL database, authentication, user profiles
- Groq Cloud — LPU inference for Whisper and Llama
- Google AI Studio — Gemini 2.5 Flash API
- Render.com — Backend hosting
- Vercel — Frontend hosting

6 · Current Status

Status	☑ Production Ready — Submitted May 2026
Live App	Link available in GitHub repository description
GitHub Access	Lab staff added as collaborator — students access via the lab
Backend	Deployed on Render.com (FastAPI + Supabase PostgreSQL)
Frontend	Deployed on Vercel (React 19 + Vite + Tailwind v4)
Android	PWA-ready; APK buildable via Android Studio + Capacitor

*This document is part of the CogniTek project handover package.
See also: [Services_Usage_Guide.md](#) · [GitHub Repository](#)*